

BIT MESRA Invigilation Duty Allocation System

End-to-End System Execution & Implementation Workflow Walkthrough

1. Server Startup & Excel Bootstrapping Sequence

The system boot lifecycle begins on the backend server side using `server.py`:

- **Roster Verification:** The server calls the `bootstrap_state()` routine on startup. It searches for `solver_state.json` in the workspace directory.
- **Excel Data Extraction:** If the json state is missing, it reads the spreadsheet file '*Faculty List (Emp. Code, Phone No. and E-mail ID).xlsx*' (specifically from the sheet '*Seniority Wise List*') as the absolute ground truth.
- **Roster Parsing & Normalization:** It maps Employee Code to id (safeguarding leading zeros), Faculty Member to name, and Designation to category (standardizing values to Professor, Associate Professor, and Assistant Professor). Spacers and header rows are discarded.
- **Persistent State Cache:** A default history record (previous load imbalance = 0.0) is built for all 45 faculty members. These rosters are combined with configured session schedules and written to `solver_state.json` (for state replication) and `sample_config.json` (active web config) before binding socketserver to Port 8080.

2. Web Page Resource Loading Lifecycle

Once the HTTP server is bound and active:

- **Asset Fetching:** The user opens `http://localhost:8080`. The browser downloads the static files: `index.html` (dashboard layouts), `index.css` (styles and colors), and `app.js` (state controller).
- **DOM Ready Callback:** The browser fires the `DOMContentLoaded` listener, which initializes navigation controls and invokes `loadConfigFromBackend()`.

3. Configuration Loading & UI Synchronization

Before the user runs an allocation, the UI must match the server state:

- **GET /api/config:** The frontend queries the backend via a GET request. The backend reads `sample_config.json` and sends back the active configuration payload.
- **UI State Alignment:** `app.js` checks and normalizes category fields. It updates dashboard widgets, displaying the 45 synced faculty members and session counts on stats cards, then triggers a silent solver execution to pre-populate the calendar grid.

4. Invoking the Scheduler Solver Engine

- **POST /api/solve:** When the user clicks **Run Allocation**, the frontend gathers all active parameters from memory and posts them in JSON format to /api/solve.
- **Dataclass Parsing:** The backend handler parses the input into Python dataclasses (AllocationInput) via the load_from_dict() mapping function and instantiates the InvigilationSolver engine.

5. Solver Optimization Pipeline (invigilation_scheduler.py)

The core solver engine executes a multi-layered optimization pipeline:

- **Category Target Calculations:** Computes expected duty weight-hours based on designation ratio multipliers (Professor = 2.0, Associate = 3.0, Assistant = 4.0), adjusted dynamically for historical load imbalances (adding +n hours if they were overloaded historically).
- **Greedy Initial Allocation:** Sorts all sessions chronologically. For each slot, it assigns the faculty member who currently has the lowest accumulated load.
- **Hard Constraint Validation:** Verifies that no assignment violates hard constraints: (1) *Max 1 shift per day per faculty*, (2) *No assignments during marked availability overrides*, (3) *No assignments during PG timetable class blocks*.
- **Local Search Swaps:** Performs iterative pairwise swaps of duties. Swaps are locked only if they reduce overall workload inequality (calculated using Gini Inequality Coefficient and Jain's Fairness Index) or resolve an unfilled slot. It halts once a local minimum is reached.

6. Dashboard Rendering Flow

- **Serialization:** The solver returns the AllocationResult dataclass. The backend serializes it using asdict() and returns it as a JSON payload to the browser.
- **Grid Update:** app.js renders each grid cell on the timetable dashboard (populating course codes, rooms, and assigned faculty names). It also updates fairness dials, workload charts, and statistics widgets.

7. Autosave Trigger & State Persistence

- **Autosave:** Every time the user interacts with settings, edits faculty details, or checks availability override boxes, the frontend immediately fires a silent POST /api/config.
- **Data Persistence:** The backend receives the payload and overwrites both sample_config.json and solver_state.json on disk. This prevents any data loss from server restarts or laptop shutdown cycles.